

Vetores

Prof. Me. Alex Paixão

Lagarto – SE

Vetores

Roteiro:

- Declaração de vetores
- Uso correto de vetores
- Vetores com tamanho variável
- Matrizes
- Vetores de caracteres (texto)

Vetores

Conceito

- Sequência de valores
- Todos do mesmo tipo
- Nome único para a variável
- Acesso por índice
- Tamanho fixo
- Numeração de 0 até tamanho-1
- Alocados sequencialmente na memória

int v[10]

v₀ (int)

v₁ (int)

v₂ (int)

v₃ (int)

v₄ (int)

v₅ (int)

v₆ (int)

v₇ (int)

v₈ (int)

v₉ (int)

Vetores

Declaração de vetor:

tipo **variável** **[tamanho]**

Qualquer tipo
de linguagem C

Nome da
variável

Números de
elementos

Exemplo:

```
int vetor[10];  
Double medidas[100];
```

int v[10]

v₀ (int)

v₁ (int)

v₂ (int)

v₃ (int)

v₄ (int)

v₅ (int)

v₆ (int)

v₇ (int)

v₈ (int)

v₉ (int)

Vetores

Declaração de vetor:

Elementos do vetor:

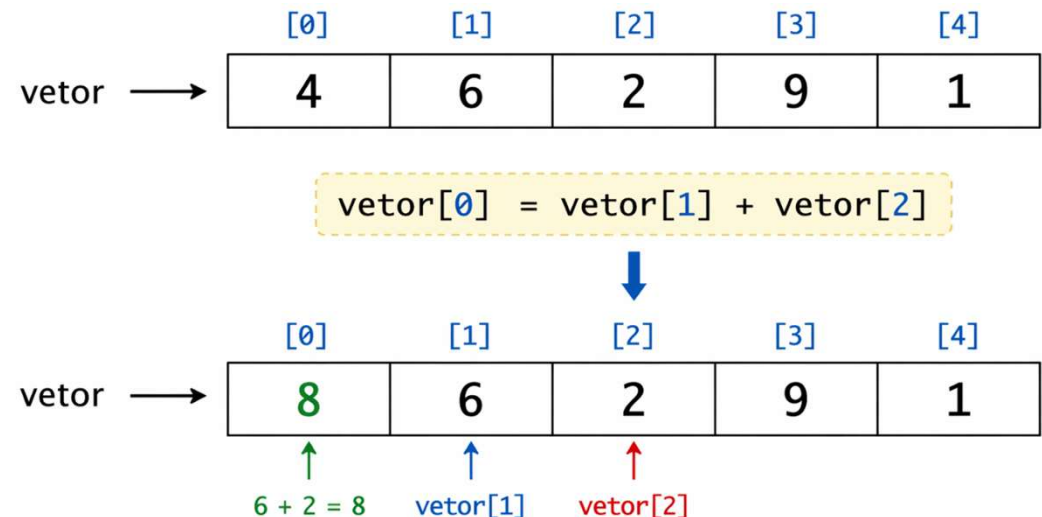
Vetor[0], vetor[1], vetor[2], ...

Atribuição:

Vetor[indice] = valor;

Exemplo:

```
int vetor[5];  
vetor[5] = 3;  
vetor[0] = vetor[1] + vetor[2];
```



Vetores

```
...  
    int vetor[10] = {20, 21, 22, 23, 24, 25, 26,  
27, 28, 29};  
    for(int i=0 ; i < 10 ; i++){  
        cout << v[i] << " ";  
    }  
    cout << "\n";  
...
```

20 21 22 23 24 25 26 27 28 29

Vetores

```
...  
int vetor[10] = {20, 21, 22, 23, 24, 25, 26,  
27, 28, 29};  
for(int i=0 ; i < 5 ; i++){  
    cout << v[i+i] << " ";  
}  
cout << "\n";  
...
```

$$0+0 = 0$$

$$1+1 = 2$$

$$2+2 = 4$$

$$3+3 = 6$$

$$4+4 = 8$$

20 22 24 26 28

0

2

4

6

8

Vetores

Declaração com conteúdo inicial:

Elementos do vetor:

tipo vetor[] = { e_1 , e_2 , $e_3 \dots$, e_{n-1} }

Exemplo:

```
int impares[] = {1, 3, 5, 7, 9};
```


Vetores

Regras para uso correto

Vetores

Cuidado com os índices

Errado:

```
int vetor[10];
```

```
...
```

```
vetor[-2] = 3;
```

```
vetor[20] = 6;
```

**Efeitos
Imprevisíveis**

Correto:

```
vetor[4] = 3;
```

```
vetor[7] = 6;
```

Vetores

Cuidado com os índices

Errado:

```
int vetor[10];  
...  
vetor = 0;
```

Correto:

```
for(int i=0 ; i<10 ; i++){  
    vetor[i] = 0;  
}
```

Vetores

Cuidado com os índices

Errado:

```
int vetorA[10], vetorB[10];  
...  
vetorA = vetorB;
```

Correto:

```
for(int i=0 ; i<10 ; i++){  
    vetorA[i] = vetorB[i];  
}
```

Vetores

Fazer um algoritmos que imprime em ordem inversa;

Matrizes

Declaração de Matrizes

Matrizes

Conceito:

- Tabela de Valores
- Acesso por dois índices: linha e coluna;
- Numeração: 0 até linhas-1; 0 até colunas-1

Outras propriedades:

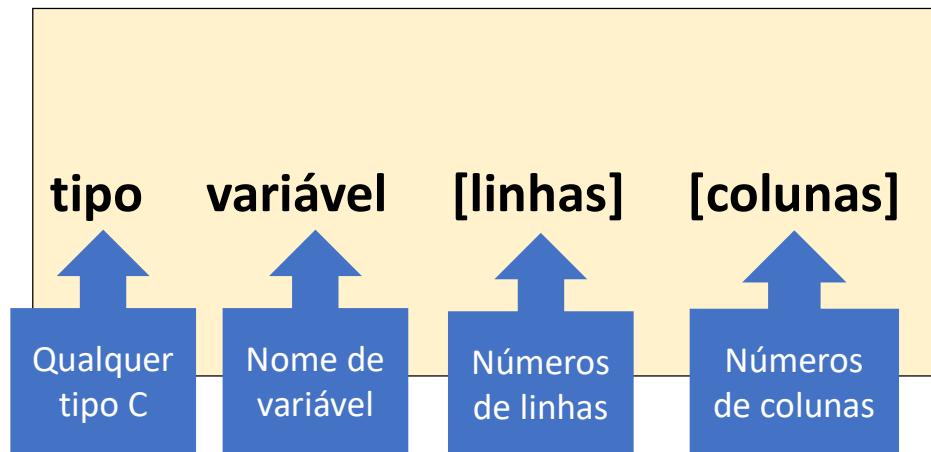
- Valores do mesmo tipo
- Acesso aos valores através de um único nome de variável
- Dimensões fixas

```
int a[4][10];
```

a_{00}	a_{01}	a_{02}	a_{03}	a_{04}	a_{05}	a_{06}	a_{07}	a_{08}	a_{09}
a_{10}	a_{11}	a_{12}	a_{13}	a_{14}	a_{15}	a_{16}	a_{17}	a_{18}	a_{19}
a_{20}	a_{21}	a_{22}	a_{23}	a_{24}	a_{25}	a_{26}	a_{27}	a_{28}	a_{29}
a_{30}	a_{31}	a_{32}	a_{33}	a_{34}	a_{35}	a_{36}	a_{37}	a_{38}	a_{39}

Matrizes

Declaração:



Exemplos:

```
int matriz[4][10];
```

```
int a[4][10];
```

a_{00}	a_{01}	a_{02}	a_{03}	a_{04}	a_{05}	a_{06}	a_{07}	a_{08}	a_{09}
a_{10}	a_{11}	a_{12}	a_{13}	a_{14}	a_{15}	a_{16}	a_{17}	a_{18}	a_{19}
a_{20}	a_{21}	a_{22}	a_{23}	a_{24}	a_{25}	a_{26}	a_{27}	a_{28}	a_{29}
a_{30}	a_{31}	a_{32}	a_{33}	a_{34}	a_{35}	a_{36}	a_{37}	a_{38}	a_{39}

Matrizes

Conceito:

Elementos do vetor:

`Vetor[0][0], vetor[0][1], vetor[0][2], ...vetor[linhas-1][colunas-1];`

Atribuição

`Vetor[linha][coluna] = valor.`

Exemplo:

```
int vetor[6][10];  
vetor[5][1] = 3;  
vetor[0][2] = vetor[1][0] + vetor[2][3];
```

Matrizes

Acesso:

Exemplo:

```
int lin, col;
int matriz[4][10];
...
for (lin = 0; lin < 4; lin++) {
    for (col = 0; col < 10; col++) {
        printf("%d ", matriz[lin][col]);
    }
    printf("\n");
}
```

Vetores multidimensionais

Tipo **variável** **[dim₁]** **[dim₂]** ... **[dim_n]** ;

Qualquer
tipo C

Nome de
variável

Dimensão 1

Dimensão 2

Dimensão n

Vetores

Vetor de caracteres (texto)

Texto

Conceito:

- **Caracteres individuais:**

- 'a', 'A', 'f', '4', '.'

← As simples

- Representam apenas um símbolo, letra ou dígito

- **Texto:**

- Sequência de caracteres

- "Algoritmos e Programação"

← As duplas

Texto

Armazenamento:

- Vetor de caracteres
- Cada caractere do texto $\Rightarrow$ um elemento do vetor
- Último caractere: nulo ('\0')
- Tamanho mínimo do vetor: comprimento do texto + 1

char[20]	U	m		t	e	x	t	o		e	m	C	\0							
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Texto

Declaração:

```
Char variável [tamanho];
```

Exemplo:

```
char texto[50];
```

Texto

Declaração:

```
char variavel [tamanho] = "texto";
```

```
Exemplo: char texto[50] = "João da Silva";
```

```
char variavel [] = "texto";
```

```
Exemplo: char texto[] = "João da Silva";
```


Texto

Ler e imprimir:

Impressão:

```
char texto[] = "Um texto em C";  
printf("A variável: %s", texto);
```

Leitura:

```
char texto[20];  
printf("Escreva uma palavra: ");  
scanf("%s", texto);
```

OBS: • Leitura por palavra

- Não usa &
- Cuidado com tamanho da variável

Texto

Atribuir à uma variável tipo texto:

Errado:

```
char texto[100];  
...  
texto = "Alex Paixao";
```

Correto:

```
#include<string.h>  
...  
strcpy(texto, "Alex Paixao")
```

Texto

Acessar caracteres:

```
char texto[20] = "Um texto em C";  
texto[12] = 'B';  
printf("%s", texto);
```

char[20]	U	m		t	e	x	t	o		e	m	C	\0							
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Um texto em B



C = C++

Vetores e Matrizes

C++

- A diferença de implementação em relação a C

Em C

```
int v[5];  
int m[3][4];  
// tamanho fixo / manual
```

Em C++

```
#include <vector>  
vector<int> v(5);  
vector<vector<int>> m(3, vector<int>(4));
```

Vetores

C++

- Em C você controla o armazenamento; em C++ o contêiner faz isso por você.

C memória manual

- Array tem tamanho fixo quando declarado estaticamente.
- Para crescer, você geralmente usa malloc/realloc/free.
- Funções recebem ponteiro + tamanho separado.

C++ contêiner pronto

- std::vector gerencia tamanho e memória.
- push_back adiciona elementos sem realloc manual.
- size() acompanha a quantidade de elementos.

Vetores e Matrizes

Vetor: declaração e preenchimento

- A comparação direta entre o que você já mostrou em C e a forma idiomática em C++.

C: array fixo

```
int v[5];

for (int i = 0; i < 5; i++){
    scanf("%d", &v[i]);
}
```

C++: vector com tamanho inicial

```
#include <vector>
#include <iostream>
using namespace std;

vector<int> v(5);
for (int i = 0; i < v.size(); i++) {
    cin >> v[i];
}
```

Vetores e Matrizes

Vetor: crescer, percorrer e retirar

- O vector simplifica as operações que em C dependem de tamanho e ponteiro.

Adicionar elementos

```
vector<int> notas;  
  
notas.push_back(8);  
notas.push_back(10);  
notas.push_back(7);  
  
cout << notas.size(); // 3
```

Percorrer sem usar índice

```
for (int x : notas) {  
    cout << x << " ";  
}  
  
// com índice, quando precisar:  
for (int i = 0; i < notas.size();  
i++)  
    cout << notas[i];
```


Vetores e Matrizes

Vetor: crescer, percorrer e retirar

- O vector simplifica as operações que em C dependem de tamanho e ponteiro.

Retirar informações do vetor

```
notas.pop_back(); // remove o último (7)
cout << notas.size(); // 2
```

Vetores e Matrizes

Matriz: de `int m[lin][col]` para vector de vectors

- C++ mantém a ideia de linha e coluna, mas permite tamanho definido em execução.

C: tamanho fixo ou VLA

```
int lin = 3, col = 4;
int m[3][4];

for (int i = 0; i < 3; i++)
    for (int j = 0; j < 4; j++)
        scanf("%d", &m[i][j]);
```

C++: matriz dinâmica

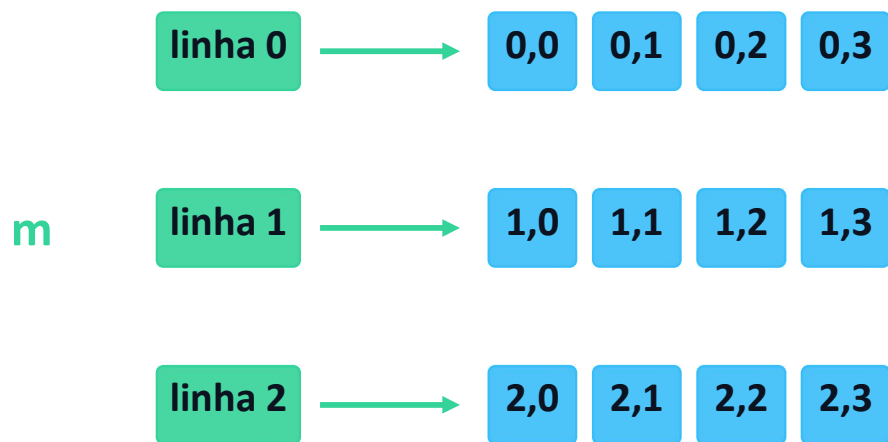
```
int lin = 3, col = 4;
vector<vector<int>> m(lin,
vector<int>(col));

for (int i = 0; i < lin; i++)
    for (int j = 0; j < col; j++)
        cin >> m[i][j];
```

Vetores e Matrizes

Matriz: visualizando o `vector<vector<int>>`

- Pense em um vector de linhas. Cada linha é outro vector.



vector externo: guarda as linhas

Criação equivalente ao desenho

```
int lin = 3, col = 4;  
vector<vector<int>> m(  
lin, vector<int>(col, 0));
```

cada linha: `vector<int>`

Vetores e Matrizes

Resumo para a transição C → C++

Trocas principais

`int v[n]` → `vector<int> v(n)`

`int m[l][c]` → `vector<vector<int>> m(l, vector<int>(c))`

`tamanho separado` → `v.size()`

`malloc/free` → `o vector gerencia memória`

`ponteiro na função` → `const vector<int> &v`

Vetores e Matrizes

Tudo Sobre Vector

Sites Principais

<https://cppreference.com/cpp/container/vector>

<https://cplusplus.com/reference/vector/>

